S3와 유사한 객체 저장소

가상 면접 사례로 배우는 대규모 시스템 설계 기초 2 · 9장

멋사 백엔드 스터디 6주차 · 임근엽

오늘의 목표

1. 어떤 데이터를 객체 저장소에 넣어야 하는가
2. 관계형 DB와는 무엇이 다른가

저장 장치 계층

위로 갈수록 빠르고 비싸다

↑ 빠름 · 비쌈



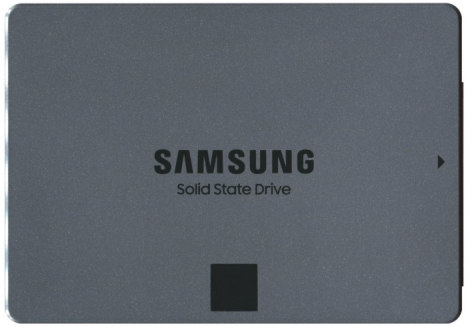
↓ 느림 · 쌈

HDD vs SSD

랜덤 접근은 수천 배, 순차 접근은 수 배 차이



HDD · 원판 + 헤드



SATA SSD



NVMe SSD (M.2)

	HDD	SATA SSD	NVMe SSD
1GB를 4KB씩 랜덤으로	~30분	~수 초	~0.5초
1GB 순차로	~4~5초	~2초	~0.2초
TB당 가격	~$11~15	~$35~60	~$30~80

사진: Evan-Amos (CC BY-SA 3.0), PantheraLeo1359531 (CC BY 4.0), User5515 (CC0) · Wikimedia Commons · 수치는 드라이브 1개 기준 대략값

개발자 입장

우리는 추상화된 걸 가져다 쓴다

개발자	S3 · MySQL · PostgreSQL
⬇	
AWS (SW)	서비스로 빌려준다
⬇	
하드웨어	SSD/HDD · 위치 · 남은 용량 · 장애

블록 추상화

디스크 종류와 무관하게, 번호 붙은 4KB 블록으로 다룬다

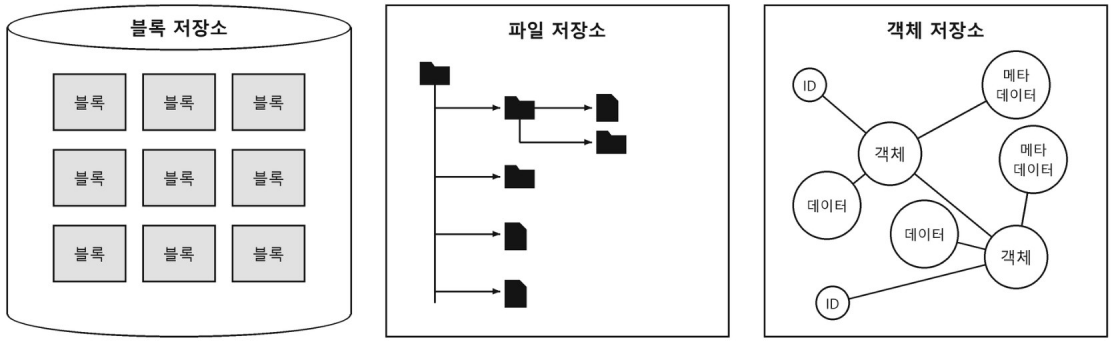
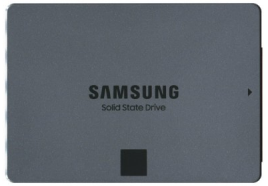


그림 9.1 저장소 유형

- 파일 시스템(inode)은 **블록 번호**만 기록
- 번호 → 실제 위치 변환은 장치가 담당 (HDD 컨트롤러, SSD FTL)



교체 + 포맷



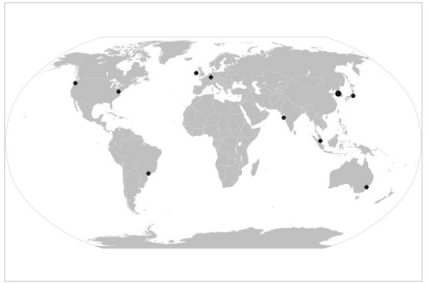
포맷 = 새 디스크에 빈 블록 목록 + inode 표 생성

`open()` · `read()` · `write()` → 코드 수정 없음

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』 · 사진: Evan-Amos (CC BY-SA 3.0), PantheraLeo1359531 (CC BY 4.0)

데이터센터 계층

장애가 퍼지는 범위(장애 도메인)로 나눈다



리전

서울, 도쿄, 버지니아 …



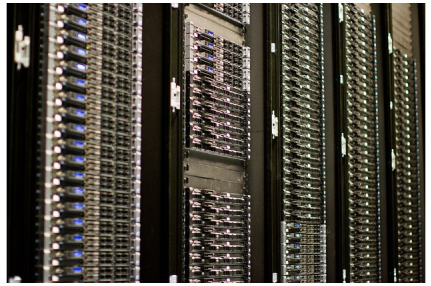
AZ

독립 전원·네트워크
정전 → AZ 전체



데이터센터

AZ 안의 건물



랙

스위치 고장 → 랙 전체



host

서버 한 대
데이터 노드 = 그 위 프로세스

데이터 노드 → 디스크 상태(생존 · 디스크 수 · 남은 용량)를 **박동 메시지**로 주기적 보고

사진: Theodore Christopher (CC0), Victor Grigas (CC BY-SA 3.0), Dsv (퍼블릭 도메인), BlankMap-World (퍼블릭 도메인) · Wikimedia Commons

데이터 조회

실제 질의는 단건보다 구간 조회가 많다

단건 조회

```
SELECT * FROM orders WHERE order_id = 1042;
```

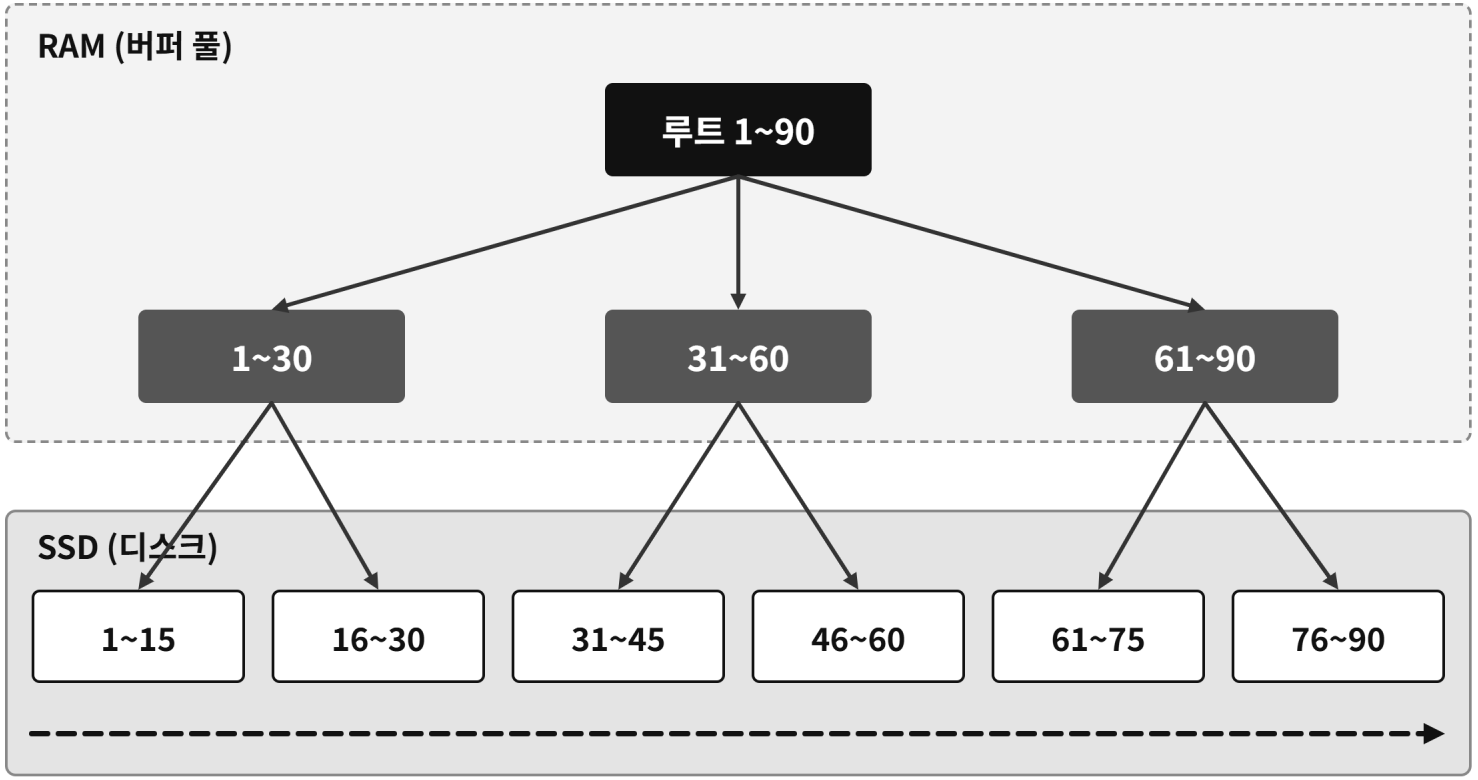
구간 조회

```
SELECT * FROM orders
WHERE created_at BETWEEN '2026-11-01' AND '2026-11-30'
ORDER BY created_at LIMIT 100;
```

인덱스	단건	구간
해시	O(1)	✕ (정렬 없음)
B+트리 (정렬)	O(log n)	◯

RDB 내부: B+트리

정렬을 디스크 위에 유지하는 자료구조



- 노드 = 16KB 페이지, 자식 수백 개
- 수억 건도 높이 **3~4**
- 리프 연결 → 구간 질의
- 페이지가 흩어져 있음 → 랜덤 I/O → 보통 SSD
- 상위 노드는 버퍼 풀에 캐시

RDB의 한계

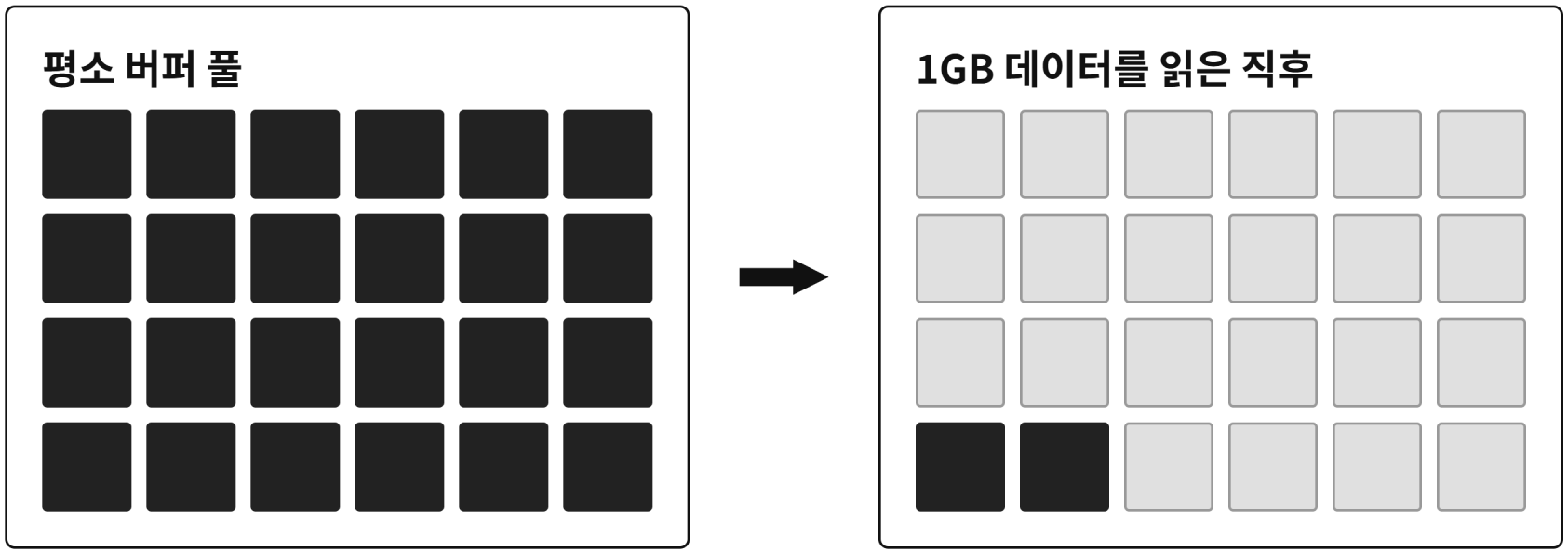
작고 균일한 데이터를 위한 저장소

1. **쓰기 서버가 한 대** — 용량을 넘으면 샤딩 → 분산 트랜잭션
2. **버퍼 풀 공유** — 큰 데이터가 캐시를 밀어냄
3. **비용** — SSD 점유, 로그·복제·백업이 함께 무거워짐

	MySQL	PostgreSQL
큰 값 한 칸 최대	4GB (LONGBLOB)	1GB
요청 한 번 최대	1GB (max_allowed_packet)	—

1GB 데이터 하나가 캐시를 밀어낸다

트리 밖으로 빼도(포인터) 같은 서버, 같은 버퍼 풀을 쓴다



65,536

1GB ÷ 16KB 페이지

■ 검정: 인덱스 페이지

□ 회색: 1GB 데이터 페이지

데이터 대신 주소(UUID)를 넣자

메타데이터와 데이터의 분리

```
key: (bucket, object_name)
→ object_id (UUID)
→ 데이터 저장소의 객체
```

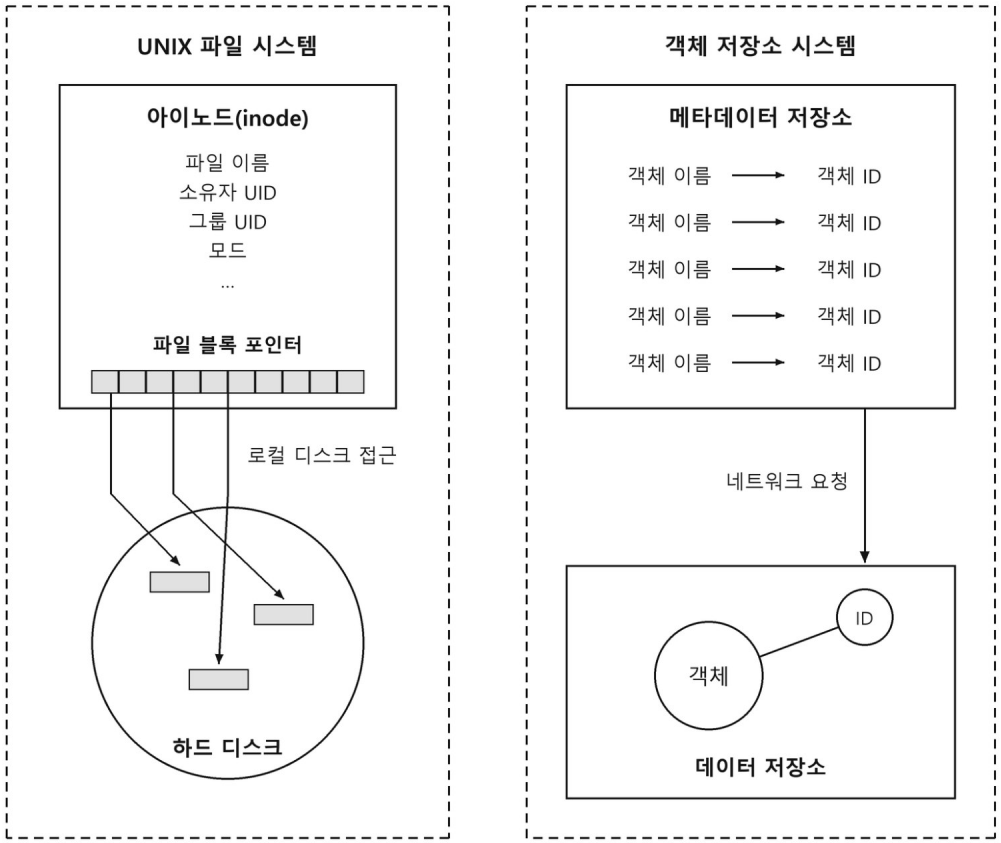


그림 9.2 UNIX 파일 시스템과 객체 저장소

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』

UUID

서버끼리 상의하지 않고 각자 만들어도 겹치지 않는 ID

```
550e8400-e29b-41d4-a716-446655440000
```

128비트 난수

그중 122비트가 랜덤
(v4 기준)

2^122 ≈ 5.3 × 10^36가지

중앙 번호표 발급기 없이
어느 서버든 바로 생성

버킷과 객체

메타데이터는 DB(SSD)에, 본체는 데이터 노드(HDD)에

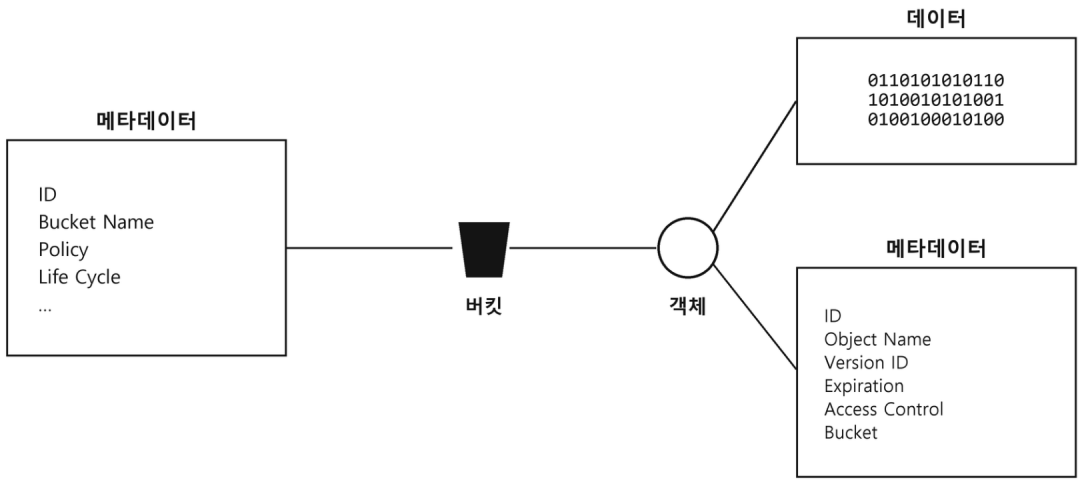


그림 9.3 버킷과 객체

	RDB 포인터	객체 저장소
가리키는 곳	같은 서버	다른 시스템
캐시·로그	공유	분리
저장 매체	SSD	HDD
용량	서버 한 대	노드 수천 대

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』

데이터 노드 내부

작은 객체들을 큰 파일 끝에 이어 붙인다 (append-only)

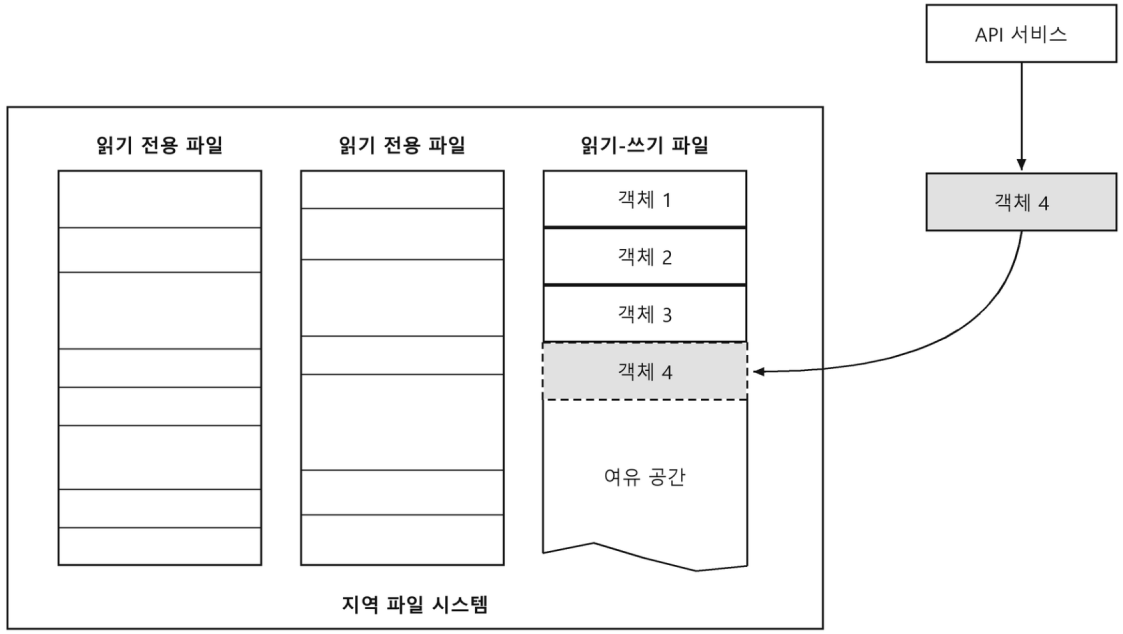


그림 9.12 작은 객체들을 한 파일에 저장하는 방안

object_mapping (노드 내 SQLite 등)

필드	의미
`object_id`	UUID
`file_name`	저장된 파일
`start_offset`	파일 내 시작 위치
`object_size`	바이트 크기

랜덤 쓰기 → 순차 쓰기 → HDD로 충분

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』

데이터 저장 흐름

배치 서비스가 가상 클러스터 지도로 저장할 노드를 고른다

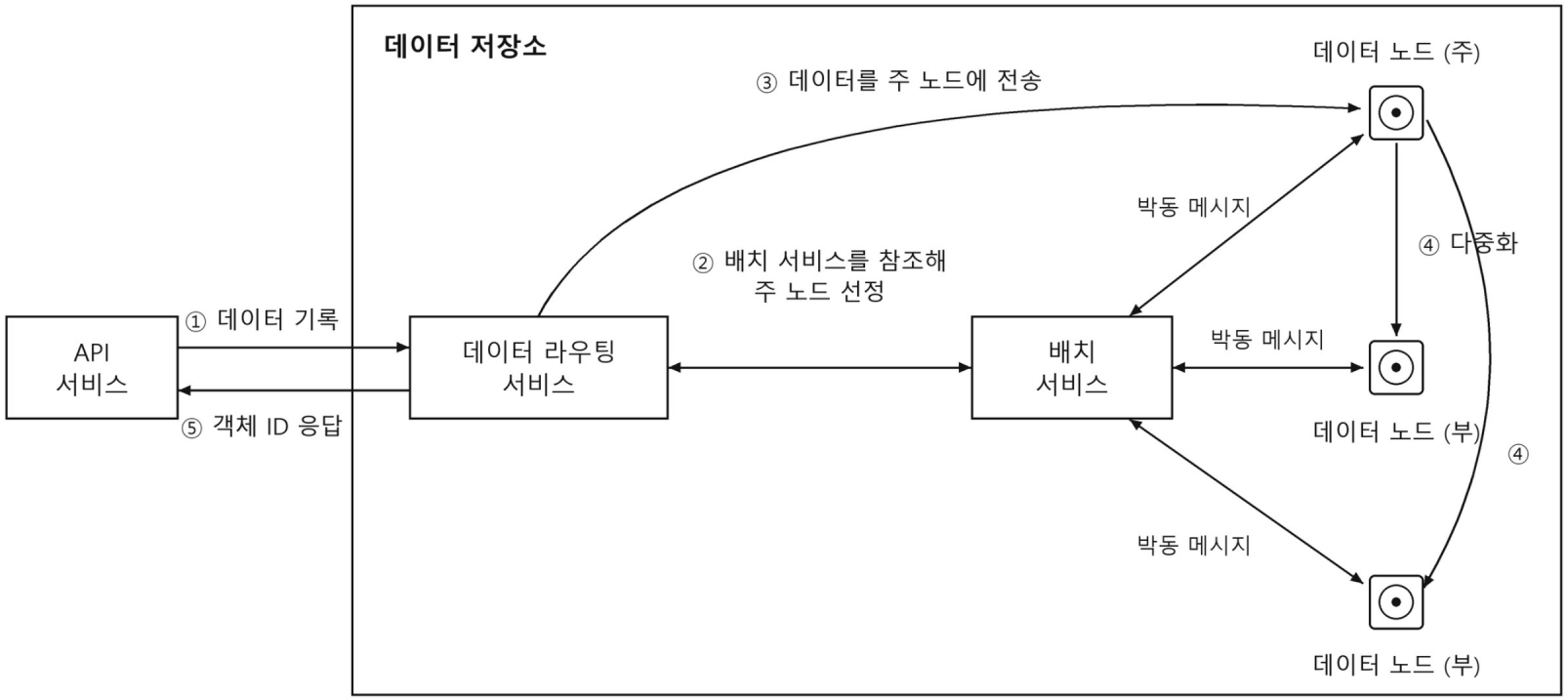


그림 9.10 데이터를 영속적으로 보관하는 흐름

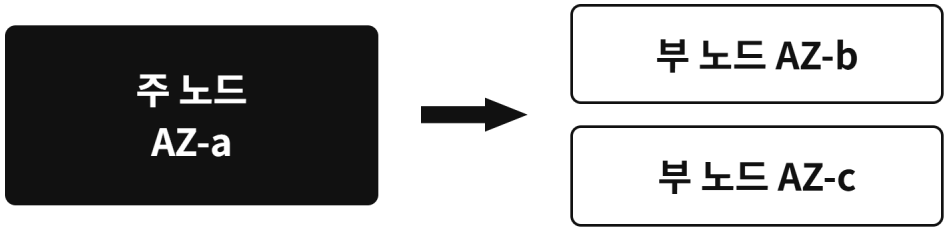
- 데이터 노드 → 박동 메시지
- 15초 유예 동안 응답 없으면 down 표시
- 주 노드 1 + 부 노드 2에 다중화

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』

장점: 비용과 내구성

순차 쓰기라서 HDD로 충분하고, 여러 노드에 나눠 담아 잃지 않는다

방법 1: 3중 복제



주 노드가 받아 부 노드 2곳에 복제 → 세 곳 모두 저장되면 성공 응답

6 nines

내구성 (HDD 연간 고장률 0.81% 가정)

3배

저장 용량

방법 2: 소거 코드 (8+4)

복제 대신 데이터를 쪼개고 패리티를 덧붙인다



D = 데이터 조각 8개, P = 패리티 4개 · 12조각을 각각 다른 노드에 · 아무거나 4개를 잃어도 남은 8개로 복원

1.5배

저장 용량 (복제 3배)

1/3

동시 손실 허용

11 nines

내구성 (복제 6 nines)

복제 vs 소거 코드

응답 속도냐, 용량·내구성이냐

	3중 복제	소거 코드 (8+4)
저장 용량	3배	1.5배
내구성	6 nines	11 nines
견디는 손실	사본 2개	조각 4개
읽기	한 노드에서 바로	최소 8노드에서 모아 + 복원 계산
적합한 경우	지연이 중요	용량·내구성이 중요

단점 1: 질의 능력

모든 질의는 메타데이터의 키 정렬에 의존한다

가능 ◯	불가 ✕
키로 객체 하나 조회	크기·날짜·태그로 검색
prefix 목록 (사전순)	다른 기준 정렬
1,000개 단위 페이지네이션	조인, COUNT 집계

S3 API

```
GET /?list-type=2&prefix=abc/
&max-keys=1000
```

메타데이터 DB에서 (책 예시)

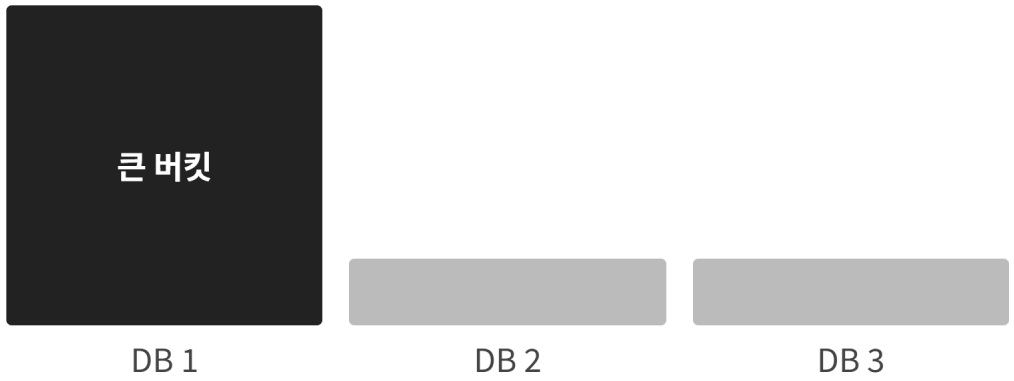
```
SELECT * FROM object
WHERE bucket_id = "123"
AND object_name LIKE 'abc/%'
```

S3에는 디렉터리가 없다 — photos/2026/a.jpg 전체가 키 하나, "폴더"는 prefix로 묶어 보여주는 것

메타데이터 샤딩과 목록 조회

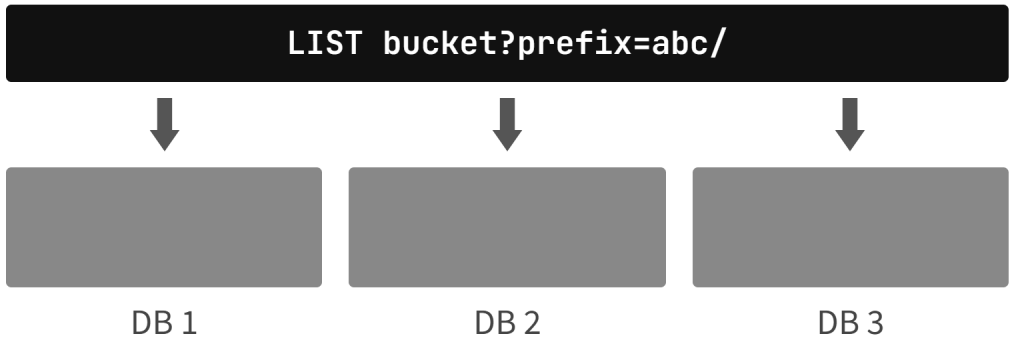
객체가 수십억 개라 메타데이터도 여러 DB로 나눠야 한다

bucket_id 기준



한 샤드에 몰림 (핫스팟)

hash(bucket_name, object_name) 기준



고르게 퍼지지만, 목록은 모든 샤드에 질의 후 병합

책의 해법: 목록 조회 전용 테이블을 bucket_id 기준으로 따로 둔다 (비정규화)

질의 능력 보완

S3 바깥에 쌓아서 해결한다

방법	내용	한계
키 설계	자주 찾는 조건을 키 앞에 uploads/user=42/2026/11/…	검색 축이 하나
외부 인덱스	업로드 이벤트 → DynamoDB · RDB · OpenSearch에 메타정보 기록	비동기라 S3와 어긋날 수 있음
AWS 기능	S3 Metadata / Inventory → Athena로 SQL 조회	실시간 아님, 추가 비용

질의 기능을 키우지 않는 것은 확장성을 지키기 위한 선택

단점 2: 수정 · 삭제와 쓰레기 수집기

삭제는 표시만 하고, 공간은 쓰레기 수집기(GC)가 몰아서 회수한다

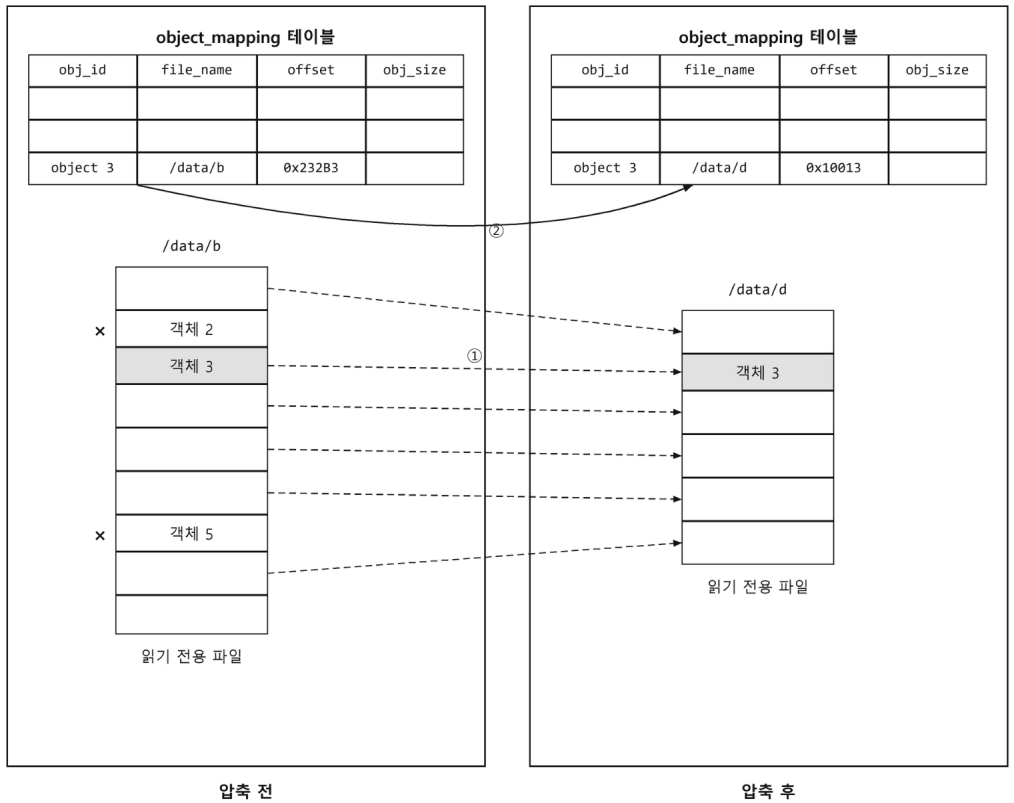


그림 9.26 쓰레기 수집기 정리 메커니즘

- 바로 지우면 뒤를 당겨야 함 → 삭제마다 O(N)
- 지연 삭제: 삭제 표시만
- 정리(compaction): 살아 있는 객체만 새 파일로 복사 → object_mapping 갱신 → 옛 파일 삭제
- 수정 = 삭제 + 새로 추가 → PATCH ×, PUT ○

GC 대상: 지연 삭제된 객체 · 갈 곳 없는 데이터(중단된 업로드) · 훼손된 데이터(체크섬 실패)

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』

멀티파트 업로드

업로드가 길수록 네트워크 장애에 걸릴 확률도 커진다

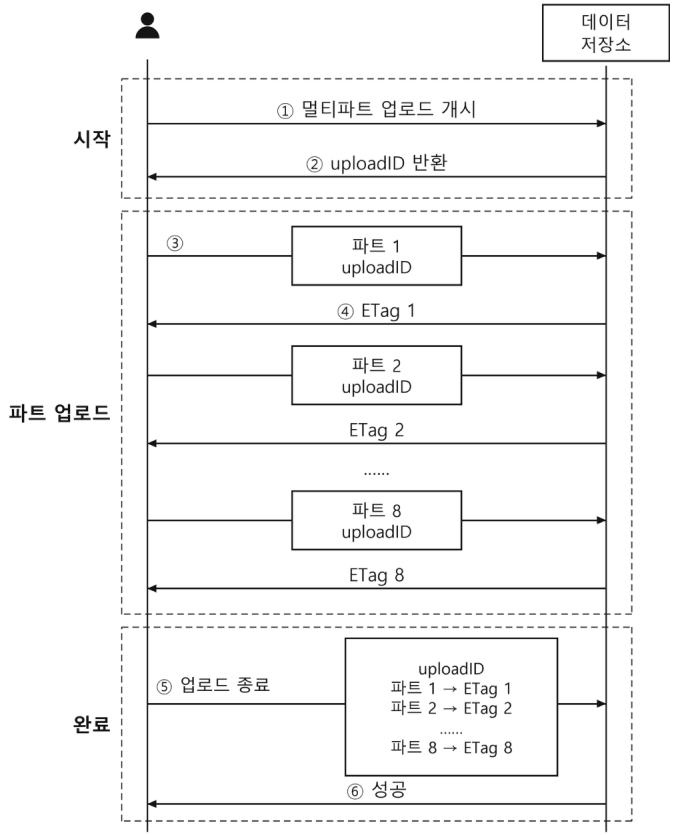


그림 9.25 멀티파트 업로드

```
T = S / B
P(끊김) = 1 - e^(-λT) ≈ λ · T
```

S: 객체 크기 · B: 대역폭 · λ: 단위 시간당 장애 발생률

	한 번에	n개 파트 병렬
실패 시 재전송	S 전체	S / n
예: 1.6GB, n = 8	1.6GB	200MB

완료 안 된 파트는 GC · 수명 주기 규칙이 정리

그림: 『가상 면접 사례로 배우는 대규모 시스템 설계 기초 2』 · S3: 파트 5MB~5GB, 최대 1만 개

정리

작고 질의가 필요한 것은 DB에,
크고 바뀌지 않는 것은 객체 저장소에.
둘은 키(UUID)로 잇는다.

저장 장치 → RDB와 B+트리 → RDB의 한계 → 메타데이터/데이터 분리 → 비용·내구성 → 질의·수정 삭제